第 10 章　图形用户界面

图形用户界面（Graphical User Interface，GUI）是程序与用户交互的方式，利用它可以接受用户的输入并向用户输出程序运行的结果。本章将介绍图形用户界面的基本组成和主要操作，包括 AWT 组件、Swing 组件、布局管理、事件处理、绘制图形、显示动画等，在本章的最后还介绍了基于 GUI 的应用程序的一般建立方法，包括使用菜单、工具栏、剪贴板等。

10.1　界面组件

10.1.1　图形用户界面概述

设计和构造用户界面，是软件开发中的一项重要工作。用户界面是计算机用户与计算机系统交互的接口，用户界面功能是否完善、使用是否方便，将直接影响到用户对应用软件的使用。图形用户界面（GUI），使用图形的方式借助菜单、按钮等标准界面元素和鼠标操作，帮助用户方便地向计算机系统发出命令，启动操作，并将系统运行的结果同样以图形的方式显示给用户。图形用户界面画面生动、操作简便，已经成为目前几乎所有应用软件的既成标准。所以，学习设计和开发图形用户界面是十分重要的。

简单地说，图形用户界面就是一组图形界面成分和界面元素的有机组合，这些成分和元素之间不但外观上有着包含、相邻、相交等物理关系，内在的也有包含、调用等逻辑关系，它们互相作用、传递消息，共同组成一个能响应特定事件、具有一定功能的图形界面系统。

Java 语言中，处理图形用户界面的类库主要是 java.awt 包和 javax.swing 包。

AWT 是 abstract window toolkit（抽象窗口工具集）的英文缩写。“抽象窗口”使得开发人员所设计的界面独立于具体的界面实现。也就是说，开发人员用 AWT 开发出的图形用户界面可以适用于所有的平台系统。当然，这仅是理想情况。实际上 AWT 的功能还不是很完全，Java 程序的图形用户界面在不同的平台上（例如，在不同的浏览器中）可能会出现不同的运行效果，如窗口大小、字体效果将发生变化等。

javax.swing 包是 JDK1.2 以后版本所引入的图形用户界面类库，swing 是功能强大的 Java 的基础类库（JFC）的一部分，其中定义的 SwingGUI 组件相对于 java．awt 包的各种 GUI 组件增加了许多功能。

设计和实现图形用户界面的工作主要有以下几点。

（1）创建组件（Component）：创建组成界面的各种元素，如按钮、文本框等。

（2）指定布局（Layout）：根据具体需要排列它们的位置关系。

（3）响应事件（Event）：定义图形用户界面的事件和各界面元素对不同事件的响应，　从而实现图形用户界面与用户的交互功能。

本章将对组件、布局、事件等进行讲解，读者可以据此编制一些图形用户界面的程序。在实际开发过程中，经常借助各种具有可视化图形界面设计功能的软件，如 NetBeans, Jbuilder，

Visual Age for Java，Java Workshop 等，这些工具软件有助于提高界面设计的效率。

10.1.2　界面组件分类

在 Java 中，构成图形用户界面的各种元素称为组件（Component）。Java 程序要显示的 GUI 组件都是抽象类 java.awt.Component 或 java.awt.MenuComponent 的子类。MenuComponent 是与菜单相关的组件，将在 10.8 节中介绍，这里介绍 Component 类。

组件分为容器（Container）类和非容器类组件两大类。容器本身也是组件，但容器中可以包含其他组件，也可以包含其他容器。非容器类组件的种类较多，如按钮（Button）、标签（Label）、文本类组件 TextComponent 等。

容器又分为顶层容器和非顶层容器两大类。顶层容器是可以独立的窗口，顶层容器的类是 Window，Window 的重要子类是 Frame 和 Dialog。非顶层容器，不是独立的窗口，它们必须位于窗口之内，非顶层容器包括 Panel 及 ScrollPane 等， Panel 的重要子类是 Applet。其中，Panel 和 Applet 的容器都是无边框的；ScrollPanel 一组是可以自动处理滚动操作的容器；Window，Frame，Dialog 和 FileDialog 是一组大都含有边框，并可以移动、放大、缩小、关闭的功能较强的容器。

AWT 组件的分类，可以用图 10-1 来表示。相关类的继承关系如图 10-2 所示。

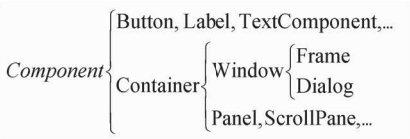


图 10-1　AWT 组件的分类

```
java.lang.Object
  |
  +-java.awt.Component
        |
        +-java.awt.Container
              |
              +-java.awt.Window
                    |
                    +-java.awt.Frame
```

```
java.lang.Object
  |
  +-java.awt.Component
        |
        +-java.awt.Container
              |
              +-java.awt.Panel
                    |
                    +-java.applet.Applet
```

图 10-2　Component 类的继承关系

注意：

Swing 组件是在 AWT 基础上开发的，大部分的 Swing 组件继承了对应的 AWT 组件，如 javax.swing.JFrame 继承了 java.awt.Frame，javax.swing.JPanel 继承了 java.awt.Panel，javax.swing.JButton 继承了 java.awt.Button， javax.swing.JLabel 继承了 java.awt.Label 等，所以本章中介绍的 AWT 组件的方法也适用于对应的 Swing 组件。

1．Container 类

Contain 类的特点是：容器中可以容纳其他组件，使用 add()方法，可以将其他对象加入到容器中，加入到容器中后，组件的位置和尺寸由布局管理器决定（关于布局管理器在 10.2 节中详细介绍）。如果要人工控制组件在容器中的大小位置，可取消布局管理器，即用方法

setLayout(null)，然后使用 Component 类的成员方法：

setLocation()，setSize()和 setBounds()来设定其大小及位置。

下面介绍 Container 的两个重要子类：Frame 及 Panel。

（1）Frame 类具有以下特点：

- Frame 类是 Window 类的直接子类；
- Frame 对象显示效果是一个“窗口”，带有标题和尺寸重置角标；
- 默认初始化为不可见的，可使用 setVisible(true)方法使之变为可见；
- 默认的布局管理器是 BorderLayout，可使用 setLayout()方法改变其默认布局管理器。

（2）Panel 类具有以下特点：

- Panel 不是顶层窗口，它必须位于窗口或其他容器之内；
- Panel 提供可以容纳其他组件，在程序中经常用于布局和定位；
- 默认的布局管理器是 FlowLayout，可使用 setLayout()方法改变其默认布局管理器；
- Panel 可以采用和所在容器不同的布局管理器。

【例 10-1】 TestFrame.java 使用 Frame。

```
import java.awt.*;
public class TestFrame {
    public static void main( String args[]) {
        Frame f = new Frame("My First Test");
        f.setSize( 170,100);
        f.setBackground( Color.blue);

        f.setVisible( true);
    }
}
```

图 10-3 使用 Frame 的运行结果

程序运行结果如图 10-3 所示。

【例 10-2】 TestFrameWithPanel.java 在 Frame 中加入 Panel。

```
import java.awt.*;
public class TestFrameWithPanel {
    public static void main(String args[]) {
        Frame f = new Frame("MyTest Frame");
        f.setSize(300,200);
        f.setLocation( 500, 400 );
        f.setBackground(Color.blue);

        Panel pan = new Panel();
        pan.setSize(150,100);
        pan.setLocation( 50, 50 );
        pan.setBackground(Color.green);

        Button b = new Button("ok");
        b.setSize( 80,20 );
        b.setLocation( 50,50 );
```

```
            b.setBackground(Color.red);

            f.setLayout(null);          // 取消默认布局管理器
            pan.setLayout( null );
            pan.add(b);                 // 面板上加入按钮
            f.add(pan);                 // 窗体上加入面板

            f.setVisible(true);
        }
    }
```

程序运行结果如图 10-4 所示。

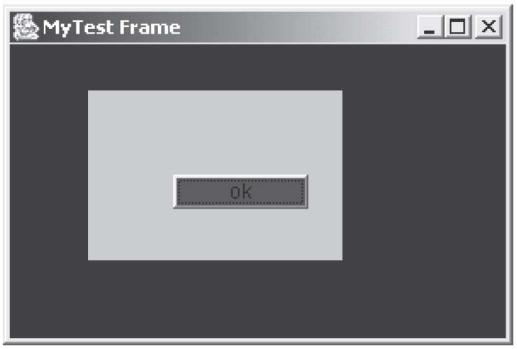


图 10-4　在 Frame 中加入 Panel

2．非 Container 类组件

非 Container 类组件，又称为控制组件，与容器不同，它里面不再包含其他组件。控制组件的作用是完成与用户的交互，包括接收用户的一个命令（如按钮），接收用户的一个文本或选择输入，向用户显示一段文本或一个图形，等等。常用的控制组件有以下几种。

（1）命令类：按钮 Button。

（2）选择类：单选按钮 JRadioButton、复选按钮 Checkbox、列表框 List、下拉框 Choice。

（3）文字处理类：文本框 TextField、文本区域 TextArea。

使用控制组件，通常需要如下的步骤。

（1）创建某控制组件类的对象，指定其文本、大小等属性。

（2）使用某种布局策略，将该控制组件对象加入到某个容器中的某指定位置处。

（3）将该组件对象注册给它所能产生的事件对应的事件监听者，重载事件处理方法，实现利用该组件对象与用户交互的功能。

10.1.3　Component 的方法

Component 类是所有组件和容器的抽象父类，其中定义了一些每个容器和组件都可能用到的方法，较常用的方法如下。

- public void add(PopupMenu popup)：在组件上加入一个弹出菜单，当用户用鼠标右键单击组件时将弹出这个菜单。

- public Color getBackground()：获得组件的背景色。
- public Font getFont()：获得组件使用的字体。
- public Color getForeground()：获得组件的前景色。
- public Graphics getGraphics()：获得在组件上绘图时需要使用的 Graphics 对象。
- public void repaint(int x，int y，int width，int height)：以指定的坐标点(x，y)为左上角，重画组件中指定宽度(width)、指定高度(height)的区域。
- public void setBackground(Color c)：设置组件的背景色。
- public void setEnabled(boolean b)：设置组件的使能状态。参数 b 为 true 则组件使能，否则组件不使能。只有使能状态的组件才能接受用户输入并引发事件。
- public void setFont(Font f)：设置组件使用的字体。
- public void setSize(int width，int height)：设置组件的大小。
- public void setVisible(boolean b)：设置组件是否可见的属性。参数 b 为 true 时，组件在包括它的容器可见时也可见；否则组件不可见。
- public void setForeground(Color c)：设置组件的前景色。
- public void requestFocus()：使组件获得注意的焦点。

由于 Component 是其他组件类的父类，所以以上所有方法都可以应用到其他各种组件中。对于具体的组件，还有相应的方法，读者可以查看 JDK 文档。

10.2 布局管理

在 Java 的 GUI 界面设计，布局控制是相当重要的一环节。将一个组件加入容器中时，布局控制决定了所加入的组件的大小和位置。如果将一个容器的布局管理器设为 null，即用方法 setLayout(null)，则要设定容器中每个对象的大小和位置。而布局管理器能自动设定容器中的组件的大小和位置，当容器改变大小时，布局管理器能自动地改变其中组件的大小和位置。

java.awt 包中共定义了五种布局管理器，每个布局管理器对应一种布局策略，分别是 FlowLayout，BorderLayout，CardLayout，GridLayout 和 GridBagLayout。下面将详细讨论这几种布局管理器。

10.2.1 FlowLayout

FlowLayout 是容器 Panel 和它的子类 Applet 默认使用的布局管理器，如果不专门为 Panel 或 Applet 指定布局管理器，则它们就使用 FlowLayout。

FlowLayout 对应的布局策略非常简单。遵循这种策略的容器将其中的组件按照加入的先后顺序从左向右排列，一行排满之后就下转到下一行继续从左至右排列，每一行中的组件都居中排列；在组件不多时，使用这种策略非常方便，但是当容器内的 GUI 元素增加时，就显得高低参差不齐。

对于使用 FlowLayout 的容器，加入组件使用如下简单的命令：

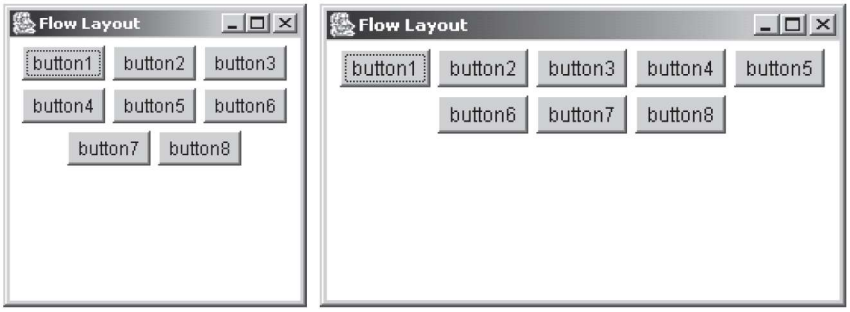
```
add(组件名);
```

设定一个容器的布局管理器，可以使用 setLayout()方法，如：

```
setLayout(new FlowLayout());
```

FlowLayout 的构造方法有三种形式：FlowLayout()，FlowLayout(int align)和 FlowLayout (int align，int hgap，int vgap)。

参数 align 指定每行组件的对齐方法，可以取三个静态常量 LEFT，CENTER，RIGHT 之一，默认为 CENTER。hgap 及 vgap 指组件间的横纵间距，默认为 5 个像素，如图 10-5 所示。



（a）　（b）

图 10-5　FlowLayout

注：（a）、（b）是当 Form 在不同宽度时的布局。

【例 10-3】 TestFlowLayout.java 使用 FlowLayout。

```
import java.awt.*;
public class TestFlowLayout {
    public static void main(String args[]) {
        Frame f = new Frame("Flow Layout");
        Button[] buttons = new Button[8];
        for( int i=0; i<buttons.length; i++)
            buttons[i] = new Button( "button"+(i+1) );

        f.setLayout(new FlowLayout(FlowLayout.LEFT, 10, 20 ));

        for( int i=0; i<buttons.length; i++)
        f.add( buttons[i] );

        f.setSize(200,200);
        // f.pack();
       f.setVisible(true);
    }
}
```

10.2.2　BorderLayout

BorderLayout 是容器 Frame 和 Dialog 默认使用的布局管理器。

BorderLayout 也是一种简单的布局策略，它把容器内的空间简单地划分为东、西、南、北、中五个区域，每加入一个组件都应该指明把这个组件加在哪个区域中。

分布在北部和南部区域的组件将横向扩展至占据整个容器的长度，分布在东部和西部的

组件将伸展至占据容器剩余部分的全部宽度，最后剩余的部分将分配给位于中央的组件。如果某个区域没有分配组件，则其他组件可以占据它的空间。例如，如果北部没有分配组件，则西部和东部的组件将向上扩展到容器的最上方，如果西部和东部没有分配组件，则位于中央的组件将横向扩展到容器的左右边界，如图 10-6 所示。

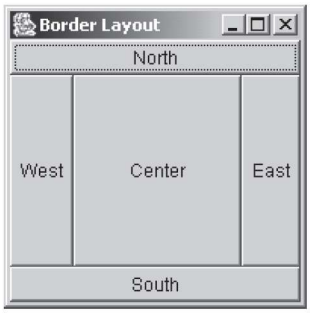


图 10-6 BorderLayout

BorderLayout 的构造方法有两种形式：BorderLayout() 和 BorderLayout(int hgap, int vgap)。

其中，hgap 及 vgap 指组件间的横纵间距，默认为 0 个像素。

只能指定五个区域位置。如果一个区域加入的组件超过多个，则只能显示该区域的最后加入的一个对象。如果容器中需要加入超过五个组件，就必须使用容器的嵌套或改用其他的布局策略。

【例 10-4】 TestBorderLayout.java 使用 BorderLayout。

```
import java.awt.*;
public class TestBorderLayout {
    public static void main(String args[]) {
        Frame f = new Frame("Border Layout");

        f.setLayout(new BorderLayout());
        f.add(new Button("North"), BorderLayout.NORTH);
        f.add(new Button("South"), BorderLayout.SOUTH);
        f.add(new Button("East"), BorderLayout.EAST);
        f.add(new Button("West"), BorderLayout.WEST);
        f.add(new Button("Center"), BorderLayout.CENTER);
        //f.add(new Button("Center"), "Center");//注意大小写
        //f.add("Center",new Button("Center") );

        f.setSize(200,200);
        f.setVisible(true);
    }
}
```

10.2.3 CardLayout

使用 CardLayout 的容器表面上可以容纳多个组件，但是实际上，在同一时间容器只能从这些组件中选出一个来显示，就像一叠“扑克牌”每次只能显示最上面的一张一样，这个被显示的组件将占据所有的容器空间。

CardLayout 的构造方法有两种形式：CardLayout() 和 CardLayout (int hgap, int vgap)。

其中，hgap 及 vgap 指组件间的横纵间距，默认为 0 个像素，如图 10-7 所示。

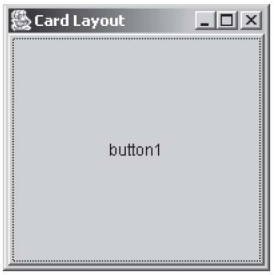


图 10-7 CardLayout

使用 CardLayout 的一般步骤如下。

（1）创建 CardLayout 对象作为布局管理器：mycard＝new

CardLayout()。

（2）使用容器的 setLayout()方法为容器设置布局管理器：setLayout(mycard)。

（3）调用容器的方法 add()将组件加入容器，同时为组件分配一个字符串的名字，以便布局管理器根据这个名字调用显示这个组件。

```
add(字符串，组件);
```

（4）调用 CardLayout 的方法 show()，根据字符串名字显示一个组件：show（容器名，字符串）；或按组件加入容器的顺序显示组件，如 first（容器名）方法显示第一个组件，last（容器名）方法显示最后一个组件等。

【例 10-5】 TestCardLayout.java 使用 CardLayout。

```
import java.awt.*;
import java.awt.event.*;
public class TestCardLayout {
    public static void main(String args[]) {
        final Frame f = new Frame("Card Layout");
        final CardLayout cl = new CardLayout();

        Button[] buttons = new Button[8];
        for( int i=0; i<buttons.length; i++){
            buttons[i] = new Button( "button"+(i+1) );
            buttons[i].addMouseListener( new MouseAdapter(){
                public void mouseClicked( MouseEvent e ){
                    if( e.getModifiers()== InputEvent.BUTTON1_MASK )
                        cl.next( f );
                    else
                        cl.previous( f );
                }
            } );
        }

        f.setLayout(cl );
        for( int i=0; i<buttons.length; i++)
            f.add( ""+(i+1), buttons[i] );

        f.setSize(200,200);
        f.setVisible(true);
    }
}
```

程序中，按钮加入了事件处理，单击鼠标左键到下一组件，单击鼠标右键到前一组件。

10.2.4　GridLayout

GridLayout 是使用较多的布局管理器，其基本布局策略是把容器的空间划分成若干行乘若干列的网格区域，组件就位于这些划分出来的小格中。GridLayout 比较灵活，划分多

少网格由程序自由控制，而且组件定位也比较精确。

使用 GridLayout 布局管理器的一般步骤如下。

图 10-8 GridLayout

（1）创建 GridLayout 对象作为布局管理器。指定划分网格的行数和列数，并使用容器的 setLayout()方法为容器设置这个布局管理器：

setLayout(new GridLayout(行数，列数))

（2）调用容器的方法 add()将组件加入容器。组件填入容器的顺序将按照第一行第一个、第一行第二个，……，第一行最后一个、第二行第一个，……，最后一行最后一个进行。每网格中都必须填入组件，如果希望某个网格为空白，可以为它加入一个空的标签 add(new Label())。如图 10-8 所示。

【例 10-6】 TestGridLayout. Java 使用 GridLayout。

```
import java.awt.*;
public class TestGridLayout {
    public static void main(String args[]) {
        Frame f = new Frame("Grid Layout");
          Button[] buttons = new Button[20];
          for( int i=0; i<buttons.length; i++)
                buttons[i] = new Button( ""+(i+1) );

          f.setLayout(new GridLayout(4,5));

          for( int i=0; i<buttons.length; i++)
                f.add( buttons[i] );

          f.setSize(200,200);
        f.setVisible(true);
    }
}
```

10.2.5 GridBagLayout

GridBagLayout 是五种布局策略中使用最复杂、功能最强大的一种，它是在 GridLayout 的基础上发展而来。因为 GridLayout 中的每个网格大小相同，并且强制组件与网格大小也相同，从而使得容器中的每个组件也都是相同的大小，显得很不自然，而且组件加入容器也必须按照固定的行列顺序，因此不够灵活。在 GridBagLayout 中，可以为每个组件指定其包含的网格个数，可以保留组件原来的大小，可以以任意顺序随意加入容器的任意位置，从而可以真正自由地安排容器中每个组件的大小和位置。但由于 GridBagLayout 的使用较复杂，限于篇幅，不再举例，读者可以查看 JDK 文档。

10.2.6 通过嵌套来设定复杂的布局

由于某一个布局管理器的布局能力有限，在设定复杂布局时，程序会采用容器嵌套的方法，即把一个容器当做一个组件加入另一个容器，这个容器组件可以有自己的组件和自己的